

Trzy zdjęcia, trzy mechanizmy

Liczby są w [poprzednim raporcie](#). Tu pokazujemy to samo na żywych zdjęciach otolitów — **ten sam obraz, różne biegi modelu**, żeby zobaczyć gołym okiem to, co kryło się za każdą kluczową liczbą w tym projekcie.

- ekspert KK (prawdziwy pierścień)
- ekspert SS (prawdziwy pierścień)
- model — finalny punkt
- metoda tradycyjna (OpenCV, bez uczenia)
- oś odczytu

Od konwencji ICES do reguły dopasowanej do obrazu

to samo zdjęcie (Z18), dwa sposoby wyznaczenia „gdzie liczyć pierścienie”

▲ największa zmierzona poprawa pojedynczego wyniku w tym zestawieniu

Automatyczne wyznaczanie osi odczytu opierało się początkowo na konwencji ICES (`find_farthest_edge`): **najdalszy punkt konturu od jądra**. Po zbudowaniu pierwszego zestawu z realnymi pozycjami pierścieni od ekspertów (zbiór „ZEGAR”) porównanie wykazało, że dla tych konkretnych zdjęć ta konwencja systematycznie wskazywała stronę bez czytelnych przyrostów. Zastąpiono ją regułą opartą wprost na obrazie: również w pełni automatycznym wyborem punktu konturu, którego profil jasności wskazuje najwięcej potencjalnych pierścieni.

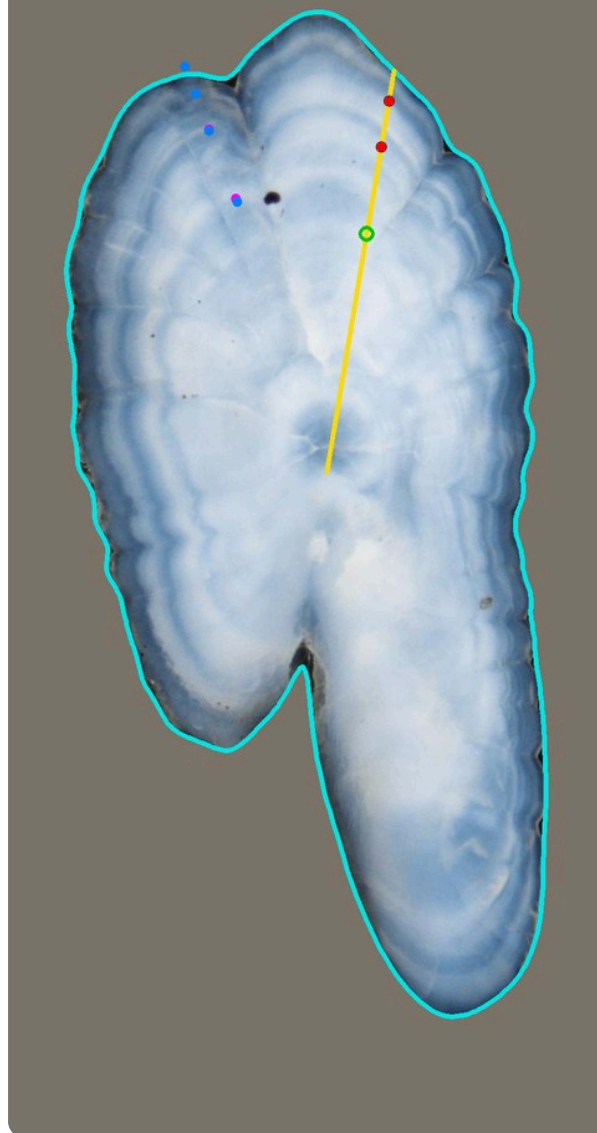
KONWENCJA ICES



1040 px

błąd wzdłuż osi odczytu

NOWA REGUŁA



8,6 px

błąd wzdłuż osi odczytu

Ta sama ryba, ten sam otolit. Żółta linia to jedyna rzecz, która się zmieniła geometrycznie — a wraz z nią wynik na tym zdjęciu poprawił się **121x**. Zmiana reguły dotyczyła wyłącznie raportowania/oceny, nigdy samego treningu (`compute_polar_grid` jest od niej niezależne) — dlatego wiek modelu się nie zmienił.

Skąd bierze się liczba w px? To nie surowa odległość na zdjęciu. Pozycja modelu i pozycja eksperta są rzutowane na tę samą, jedną linię — oś wykrytą automatycznie przez pipeline (nie na jakąś osobną „oś eksperta”) — a różnica ich położenia na tej linii jest mnożona przez długość osi w pikselach. Dlatego, gdy ta wspólna oś wskazuje w złą stronę (jak w wariantcie „konwencja ICES”), rzutowanie na nią może dać liczbę większą niż fizyczny rozmiar samego otolitu — dokładnie to widać przy 1040 px na zdjęciu powyżej.

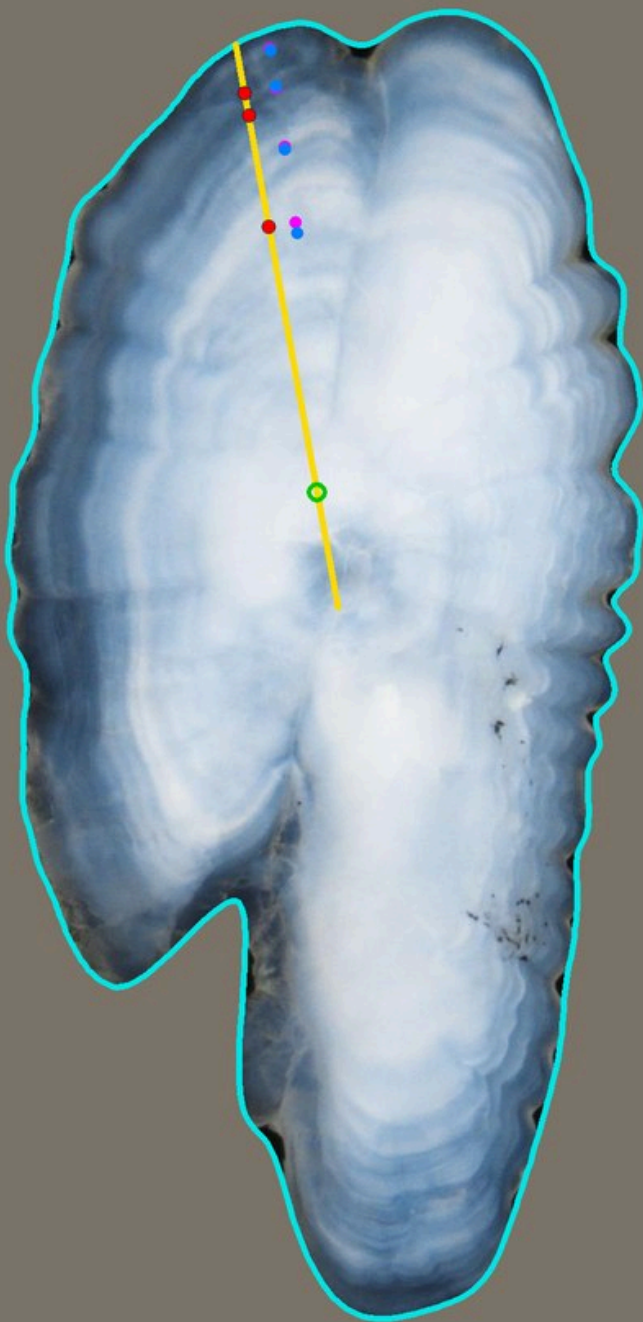
Model przewyższa dokładnością metodę tradycyjną

ten sam obraz (Z25): czerwone punkty modelu vs zielony okrąg metody tradycyjnej

✓ 27× dokładniej niż metoda tradycyjna na tym zdjęciu

Po zmianie reguły wyboru osi i przejściu na architekturę `radial_attention` (kodowanie pozycyjne promień $\times$ kąt + lokalna uwaga) głowica density **po raz pierwszy w projekcie naprawdę się nauczyła**. Metoda tradycyjna (wykrywanie krawędzi jasności wzdłuż osi, bez uczenia) trafia tu w jeden przypadkowy punkt blisko jądra. Model trafia w cztery z pięciu prawdziwych pierścieni niemal idealnie.

RUN N



11,1 px

model

296,3 px

metoda tradycyjna

Widać to bez czytania liczb: cztery czerwone kropki leżą tuż obok magenta/niebieskich punktów obu ekspertów. Zielony okrąg (metoda tradycyjna) siedzi samotnie na środku, daleko od jakiegokolwiek prawdziwego pierścienia.

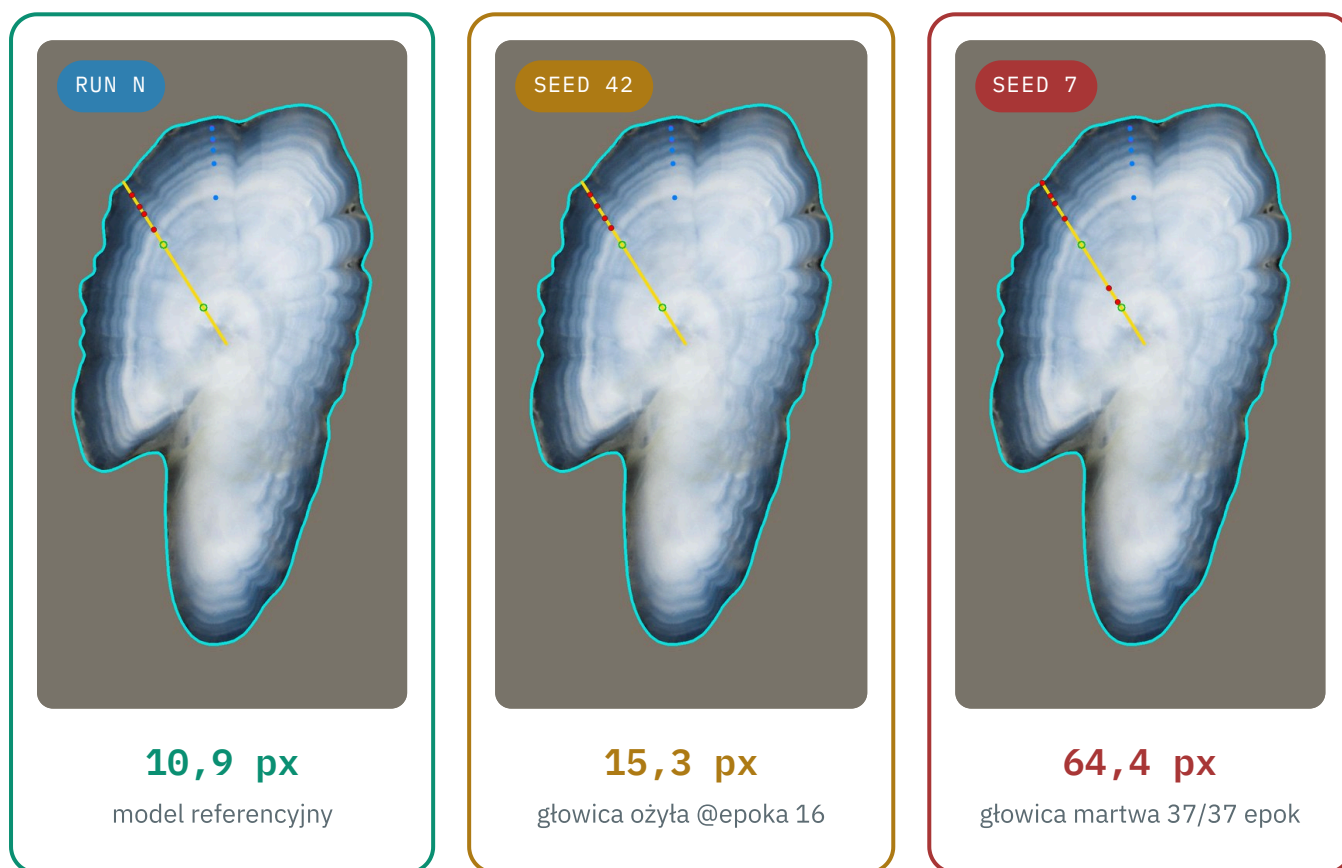
Ten sam kod, ten sam obraz

— inny wynik inicjalizacji

trzy oceny tego samego zdjęcia (Z09): Run N, Opcja A/seed 42, Opcja A/seed 7

⚡ jedyna różnica: ziarno losowości przy starcie treningu

Opcja A dokłada 30 prawdziwych pozycji pierścieni do treningu głównego modelu. Dwa biegi, identyczne poza jedną liczbą (seed): jeden „obudził” się normalnie, drugi — nigdy. Na obrazie widać to jako **jeden dodatkowy, błędny czerwony punkt**, którego seed 7 nie powinien był postawić.



Seed 42 (środek) jest niemal nierozróżnialny od Run N — te same cztery punkty, w tych samych miejscach. Seed 7 (prawa) dostawia piąty, fałszywy punkt bliżej jądra — ślad głowicy, która przez cały trening nigdy się nie nauczyła, gdzie naprawdę leżą pierścienie, i w gruncie rzeczy zgaduje.

Trzy zdjęcia, trzy różne przyczyny: geometria (oś), architektura (density head), i czysty przypadek (seed). Pełne liczby, cała historia dziesięciu prób i lista alternatyw — w [głównym raporcie zamykającym](#).

pełna kronika (38 sekcji) → outputs/DINO_proces.md

metodologia ZEGAR / oś odczytu → plans and summaries/12.08_ZEGAR_ANOTACJE_TO_D0.md